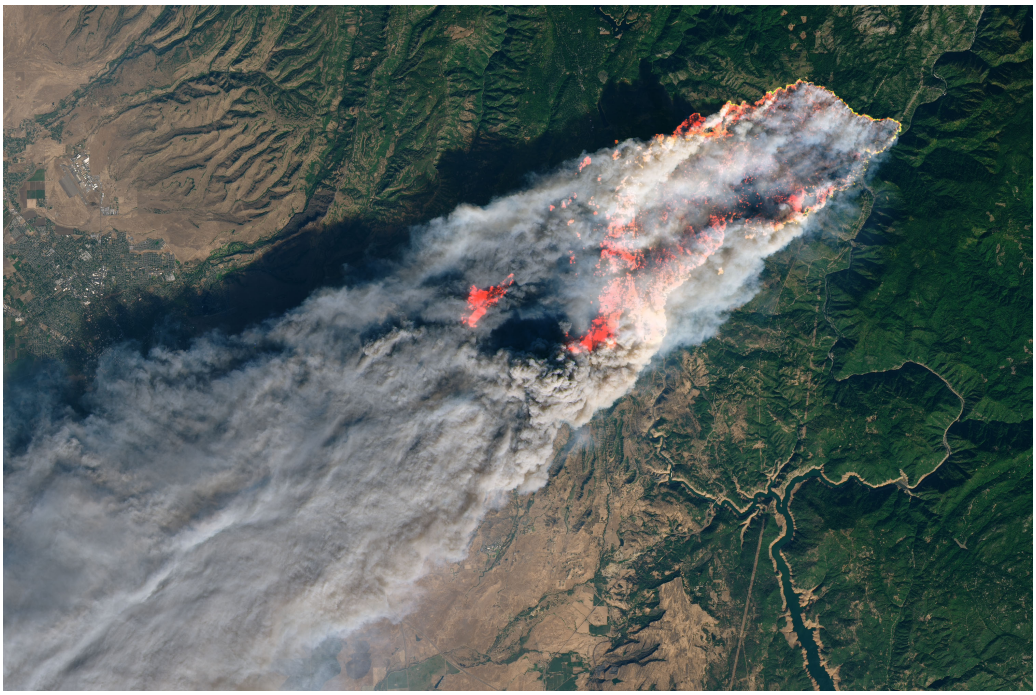
Symulacja pożaru Camp Fire (2018) w IGA-ADS

Rafał Szepieniec, Jakub Kotara

4 listopada 2026

1 Wybrany pożar: Camp Fire

Camp Fire wybuchł 8 listopada 2018 r. ok. 6:25 w kanionie rzeki Feather koło Pulga (hrabstwo Butte, Kalifornia). Przyczyną była awaria linii przesyłowej PG&E. Silny, suchy wiatr z północnego wschodu pchał ogień na południowy zachód. W ciągu kilku godzin pożar przeszedł przez Concow i Magalię i dotarł do Paradise. Ogień przeskakiwał kaniony dzięki żarowi niesionemu wiatrem. Spłonęło ok. 620 km^2 i ok. 18 800 budynków, a zginęło 85 osób. Był to najtragiczniejszy pożar w historii Kalifornii. Pożar trwał do 25 listopada (trwał 17 dni), jednak my w naszej symulacji skupiliśmy się na pierwszych godzinach trwaniu pożaru.

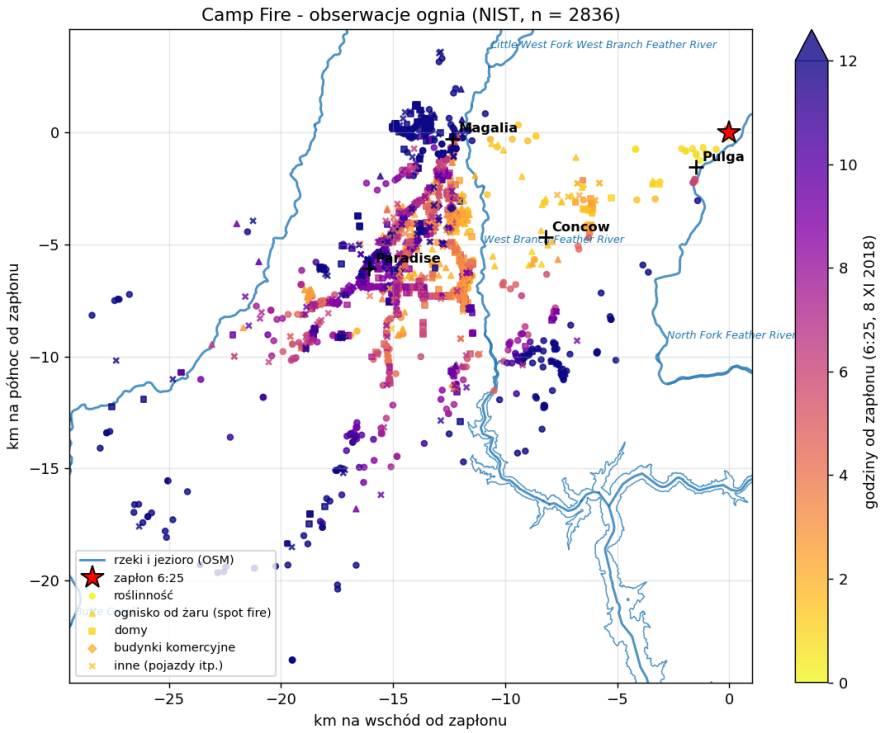


Rysunek 1: Camp Fire (By NASA, Joshua Stevens - https://earthobservatory.nasa.gov/images/144225/camp-fire-rages-in-california, Public Domain, https://commons.wikimedia.org/w/index.php?curid=74269291).

Przebieg pożaru w czasie pochodzi z raportu NIST TN 2135 [1]: 2836 obserwacji ognia z godziną i współrzędnymi GPS (rys. 2).

2 Model i dane

Obszar modelu $[0, 100]^2$ to kwadrat 30×30 km wokół zapłonu (1 jednostka = 300 m, x na wschód, y na północ). Czas $t \in [0, 10]$ odpowiada godzinom 6:25–11:25 (1 jednostka = 30 min). Siatka ma 80×80 elementów, splajny są stopnia 2, a krok czasu $\Delta t = 10^{-3}$ (10 000 kroków jawnej metody Eulera).



Rysunek 2: Obserwacje ognia NIST, kolor oznacza liczbę godzin od zapłonu. Rzeki z OpenStreetMap.

Równania z zajęć (temperatura T, paliwo F, wiatr $\mathbf{b}$):

$$\rho c_p \frac{\partial T}{\partial t} = R_c - \rho c_w \, \mathbf{b} \cdot \nabla T + \nabla \cdot \left((\kappa + 4\sigma\varepsilon\delta_x T^3) \nabla T \right) + \xi (T_0 - T) + \frac{\sigma\varepsilon}{\delta_z} (T_0^4 - T^4),$$

$$\frac{\partial F}{\partial t} = -\beta \, r F, \qquad r = A_r T e^{-T_a/T} \text{ gdy } T > T_{ig} \text{ i } F > F_{\min}, \text{ inaczej } 0,$$

$$R_c = -I(x,y) \cdot 10^4 \rho c_h h_c \frac{M}{M_1} \, r.$$

Dane wejściowe:

- paliwo: LANDFIRE 2016 FBFM40, stan sprzed pożaru [2], oraz rzeki z OpenStreetMap [3],
- wiatr: stacja RAWS Jarbo Gap (JBGC1), pomiary co godzinę z archiwum IEM [4],
- zapłon: pierwsza obserwacja NIST.

Skrypt `przygotuj_model.py` przelicza te dane do pliku `camp_fire_data.hpp` (mapa klas paliwa 201×201, tabela wiatru, punkt zapłonu).

3 Zmiany w kodzie

Zmienione pliki to `fire.hpp` i `fire.cpp` (pełny kod w dodatku A). Wszystkie parametry i wagi paliwa są w strukturze `fire_params`. Można je zmieniać bez kompilacji, np. `fire 80 2 8 TU=0.6 cw=1.0`.

```
struct fire_params {
    double rad = 0.16;      // promieniowanie: delta_x = rad / eps; prędkość frontu rośnie jak sqrt(rad)
    double loss = 9;        // straty ciepła w górę: delta_z = loss * eps (mniej = większe straty)
    double cw = 0.5;        // jak mocno wiatr niesie ciepło
    double Tig = 800;       // temperatura zapłonu
    double fuel_min = 0.2;  // od ilu paliwa komórka się pali
    double heat = 1.0 / 6;  // mnożnik ciepła spalania Rc
    double burn = 50;       // tempo wypalania paliwa
    int fuel_heat = 1;      // 1 = ciepło spalania razy waga klasy paliwa (krzewy grzeją mocniej niż podszyt)
    double dt = 1e-3;
```

```
    double t_end = 10;      // 10 = 11:25
    int out_every = 250;    // zapis .data co tyle kroków (250 = ok. 40 klatek do animacji)

    // Wagi paliwa 0..1 dla klas LANDFIRE (mapa klas w camp_fire_data.hpp, podgląd: model_paliwo.png).
    // Pali się tylko paliwo > fuel_min; więcej paliwa = dłużej płonie. Przy fuel_heat = 1 waga
    // klasy mnoży też ciepło spalania Rc.
    // Przy heat = 1/6 ogień sam się utrzymuje od wagi ok. 0.55; niżej gaśnie (chyba że sąsiedzi grzeją).
    double SH = 1.00;     // krzewy (chaparral)
    double GS = 1.00;     // trawy z krzewami
    double GR = 0.80;     // trawy
    double TU = 0.50;     // podszyt leśny: wg NIST ogień go omijał (29% obszaru, 5% obserwacji)
    double TL = 0.90;     // ściółka leśna: sosny na grzbiecie Paradise, 45% obserwacji NIST
    double SB = 1.00;     // pozostałości po wycince
    double urban = 0.90;  // zabudowa: domy w Paradise i Magalia spłonęły
    double agri = 0.15;   // pola uprawne (nawadniane)
    double rock = 0.05;   // skały, piach
    double water = 0.0;   // woda (jezioro Oroville)
    double snow = 0.0;
    double other = 0.50;  // kody spoza listy
    double river = 0.55;  // pas 150 m przy rzece: min(waga klasy, river)

    double fuel_weight(unsigned char code) const {
        double w = other;
        switch (code & ~camp_fire::RIVER) {
        case camp_fire::URBAN: w = urban; break;
        case camp_fire::SNOW: w = snow; break;
        case camp_fire::AGRI: w = agri; break;
        case camp_fire::WATER: w = water; break;
        case camp_fire::ROCK: w = rock; break;
        case camp_fire::GR: w = GR; break;
        case camp_fire::GS: w = GS; break;
        case camp_fire::SH: w = SH; break;
        case camp_fire::TU: w = TU; break;
        case camp_fire::TL: w = TL; break;
        case camp_fire::SB: w = SB; break;
        default: break;
        }
        return code & camp_fire::RIVER ? std::min(w, river) : w;
    }
};
```

Stan początkowy. Gorący obszar (1500 K w promieniu 0,75 km, spadek do 300 K w promieniu 2,1 km) w miejscu pierwszej obserwacji NIST.

```
    // Zapłon w miejscu pierwszej obserwacji NIST (6:25), gorący rdzeń ok. 0,75 km, otoczka do 2 km.
    double init_state(double x, double y) {
        double d = std::hypot(x - camp_fire::IGNITION_X, y - camp_fire::IGNITION_Y);
        return T0 + Tcomb * falloff(2.5, 7.0, d);
    }
```

Mapa paliwa. Waga klasy LANDFIRE w danym punkcie, z interpolacją dwuliniową między węzłami siatki 201×201. W pasie 150 m wzdłuż rzek brana jest mniejsza z wag klasy i `river`.

```
    // Paliwo z siatki klas LANDFIRE: waga klasy z fire_params, interpolacja dwuliniowa.
    double fuel_map(double x, double y) const {
        const int n = camp_fire::FUEL_N;
        double gx = std::clamp(x / 100.0 * (n - 1), 0.0, n - 1.0);
        double gy = std::clamp(y / 100.0 * (n - 1), 0.0, n - 1.0);
        int i = std::min(static_cast<int>(gx), n - 2);
        int j = std::min(static_cast<int>(gy), n - 2);
        double hx = gx - i;
        double hy = gy - j;
        auto f = [this, n](int a, int b) { return par.fuel_weight(camp_fire::FUEL_CLASS[b * n + a]); };
        return (1 - hx) * (1 - hy) * f(i, j) + hx * (1 - hy) * f(i + 1, j)
             + (1 - hx) * hy * f(i, j + 1) + hx * hy * f(i + 1, j + 1);
    }
```

Wiatr. Liniowa interpolacja w czasie pomiarów z Jarbo Gap. Dane są z jednej stacji, więc wiatr jest taki sam w całym obszarze. Przelicznik to 0,85 jednostki modelu na 1 m/s.

```
// Wiatr ze stacji Jarbo Gap: jedna stacja, więc taki sam w całym obszarze.
// Interpolacja liniowa między pomiarami co godzinę. Wieje z NE (ok. 30-40 st.),
// czyli pcha ogień na południowy zachód; najsilniej rano, słabiej w południe.
void wind_velocity(double /*x*/, double /*y*/, double t, double& bx, double& by) const {
    bx = interp_table(camp_fire::WIND_T, camp_fire::WIND_BX, camp_fire::WIND_N, t);
    by = interp_table(camp_fire::WIND_T, camp_fire::WIND_BY, camp_fire::WIND_N, t);
}
```

Prawa strona równania. Wprowadziliśmy trzy zmiany względem wersji z zajęć:

- ciepło spalania jest mnożone przez $I = \texttt{heat} \cdot w(x, y)$, gdzie w to waga klasy paliwa (`fuel_heat = 1`), więc krzewy grzeją mocniej niż podszyt;
- temperatura jest obcinana od dołu do 0, bo bez tego oscylacje splajnów przy zapłonie dają NaN w całym polu;
- progi i współczynniki pochodzą z `fire_params`.

```
            double bx, by;
            wind_velocity(x, y, t, bx, by);
            double intensity = par.heat * (par.fuel_heat ? fuel_map(x, y) : 1.0);

            value_type u = eval_fun(u_prev, e, q);
            value_type fuel = eval_fun(fuel_prev, e, q);

            for (auto a : dofs_on_element(e)) {
                auto aa = dof_global_to_local(e, a);
                value_type v = eval_basis(e, q, a);
                double inv = 1.0 / (rho * cp);

                // Spline ringing near the ignition can drive T below 0. pow() and
                // exp(-Ta/T) then overflow, and 0*inf becomes NaN for the whole field.
                double T = std::max(u.val, 0.0);
                double delta = T > Tig && fuel.val > par.fuel_min ? 1.0 : 0.0;
                double hc = -70; // enthalpy
                double r = 0.0;
                if (delta > 0.0)
                    r = Ar * T * std::exp(-Ta / T);
                double Rc = - intensity * 1e4 * rho * ch * hc * M / M1 * r;
                double Qw = - rho * cw * (bx * u.dx + by * u.dy);
                double qc = - kappa * grad_dot(u, v);
                double qd = 0.0; // omitted
                double qr = -4 * sigma * eps * delta_x * std::pow(T, 3) * grad_dot(u, v);
                double Qconv = xi * (T0 - T);
                double Qrz = sigma * eps / delta_z * (std::pow(T0, 4) - std::pow(T, 4));

                double val = (Rc + Qw + Qconv + Qrz) * v.val + qc + qd + qr;
                U(aa[0], aa[1]) += (u.val * v.val + steps.dt * inv * val) * w * J;

                double fval = - delta * par.burn * r * fuel.val * v.val;
                F(aa[0], aa[1]) += (fuel.val * v.val + steps.dt * fval) * w * J;
```

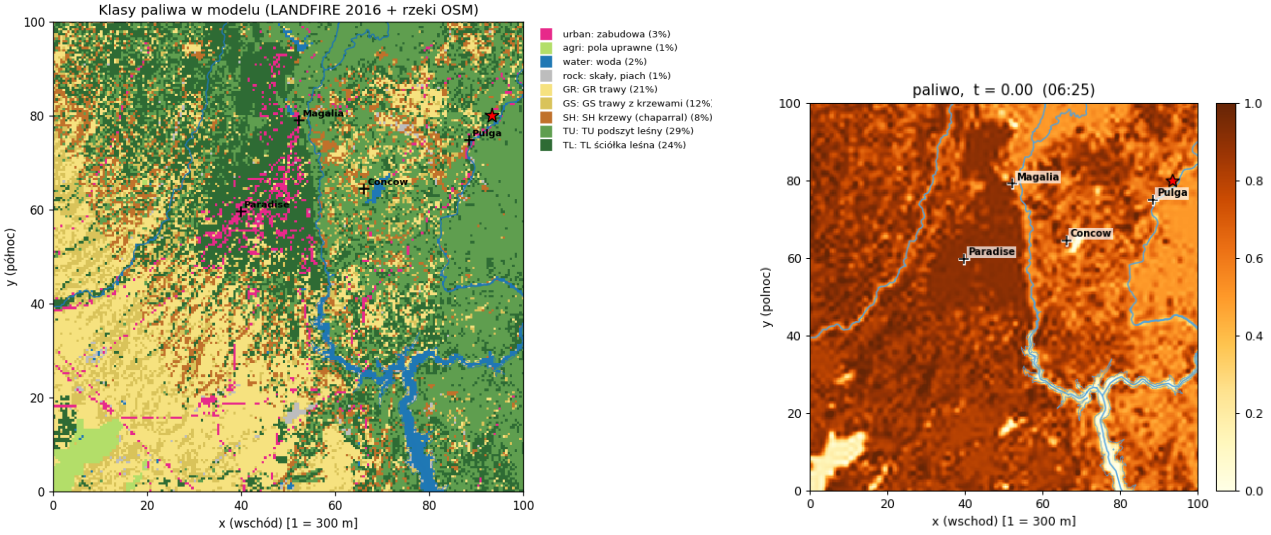
Szerokość frontu. Przy stałych z zajęć front ognia jest węższy (ok. 0,3 jednostki) niż element siatki (1,25). Ogień wtedy zatrzymuje się na liniach siatki: idzie wzdłuż osi albo gaśnie. Dlatego pomnożyliśmy `rad` (δ_x) i `loss` (δ_z) przez 6, a `heat` i β (`burn`) podzieliliśmy przez 6. Spalanie w punkcie trwa wtedy 6 razy dłużej, progi zapłonu i gaśnięcia się nie zmieniają, a front jest ok. 6 razy szerszy. Dla mnożników 3 i 6 wyniki są zbliżone.

4 Mapa paliwa

Wagi dobraliśmy, porównując wyniki symulacji z obserwacjami NIST. Podszyt (TU) zajmuje 29 % obszaru, ale ma tylko 5 % obserwacji ognia, dlatego jego waga leży poniżej progu samodzielnego palenia (ok. 0,55). Ogień przechodzi przez podszyt tylko wtedy, gdy obok palą się trawy lub krzewy. Ściółka sosnowa (TL) na grzbiecie Paradise i zabudowa mają większość obserwacji. Obserwacje NIST pochodzą jednak głównie z okolic domów i dróg.

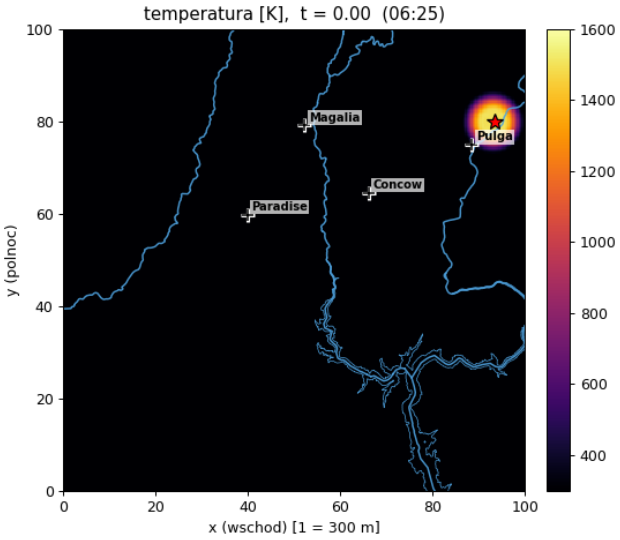
Klasa	Opis	Waga	Obszar	Obserwacje NIST
SH	krzewy (chaparral)	1.00	8 %	1 %
GS	trawy z krzewami	1.00	12 %	5 %
GR	trawy	0.80	21 %	5 %
TU	podszyt leśny	0.50	29 %	5 %
TL	ściółka leśna	0.90	24 %	45 %
urban	zabudowa	0.90	3 %	38 %
agri, rock, water	pola, skały, woda	0.15, 0.05, 0	4 %	< 1 %
river	pas przy rzece	0.55	4 %	–

Tabela 1: Wagi paliwa. Ostatnie kolumny: udział klasy w obszarze i w obserwacjach ognia NIST do 11:25.



Rysunek 3: Klasy paliwa LANDFIRE w obszarze modelu (lewy) i `fuel_init` po rzutowaniu na splajny (prawy).

5 Stan początkowy



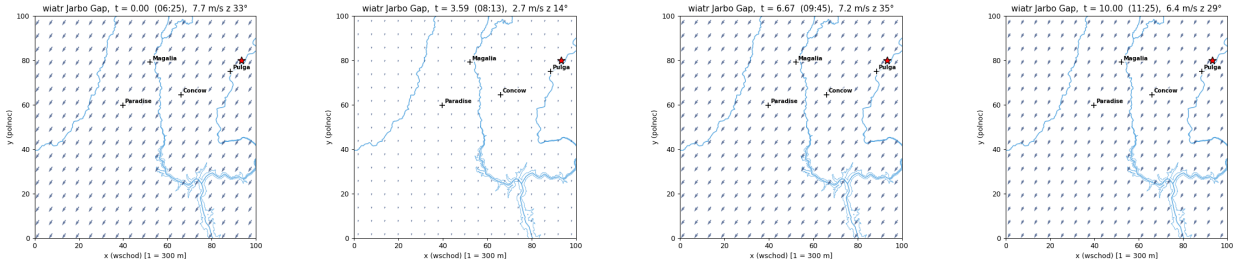
Rysunek 4: Temperatura w chwili $t = 0$ (6:25). Gwiazdka oznacza zapłon.

6 Wiatr

Godzina	5:13	6:13	7:13	8:13	9:13	10:13	11:13	12:13	13:13
Prędkość [m/s]	13.4	8.1	6.3	2.7	6.3	8.1	6.3	7.2	5.4
Skąd wieje [°]	42	33	33	14	33	37	29	31	38

Tabela 2: Pomiary wiatru w Jarbo Gap, 8 XI 2018 (czas PST).

Wiatr cały czas wieje z północnego wschodu. Najsilniejszy jest przed zapłonem, a ok. 8:00 na krótko słabnie.

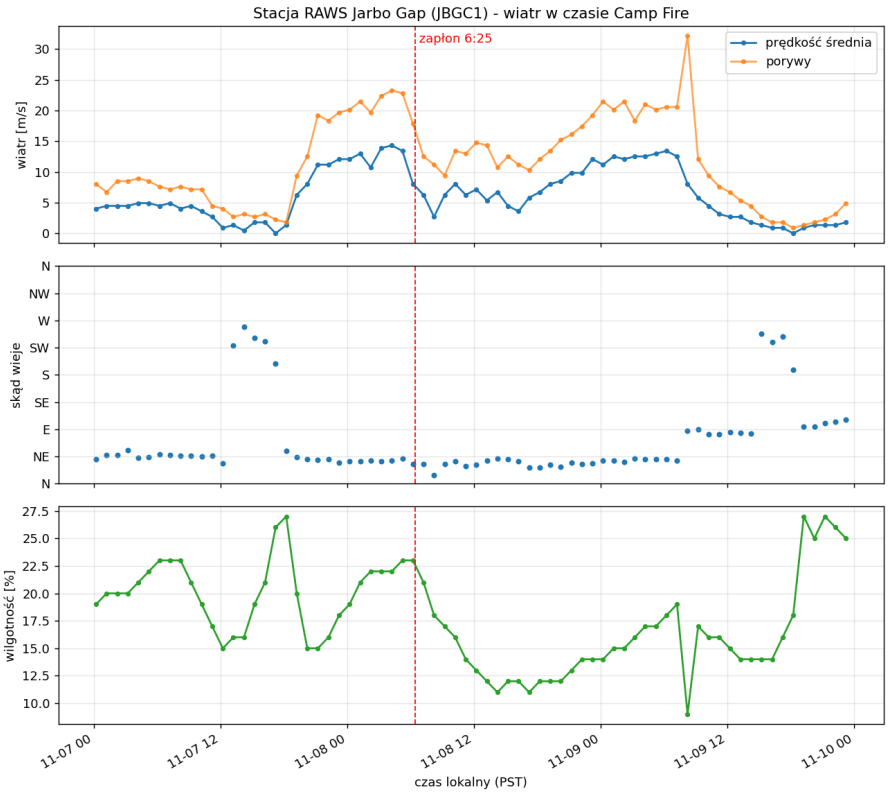


Rysunek 5: Wiatr w modelu w chwilach $t = 0$, 3,6, 6,7 i 10 (6:25, 8:12, 9:45, 11:25).

7 Animacje

Animacje są w osobnych plikach GIF w katalogu `anim/`:

- `anim/fire.gif`: temperatura,
- `anim/fuel.gif`: paliwo,
- `anim/wind.gif`: wiatr,
- `anim/pozar_nist.gif`: rzeczywisty przebieg z obserwacji NIST.



Rysunek 6: Pomiary ze stacji Jarbo Gap z 7–10 XI 2018.

8 Kroki czasowe

Ogień rusza z wiatrem na południowy zachód, wzdłuż korytarza traw i krzewów do Concow. Ok. 8:25 front stoi na rzece West Branch, a potem przechodzi na grzbiet z Magalią i Paradise. Podszyt na wschód i południe od zapłonu prawie się nie pali. Po 10:00 ogień rozchodzi się po trawach na zachodzie.

9 Porównanie z rzeczywistym pożarem

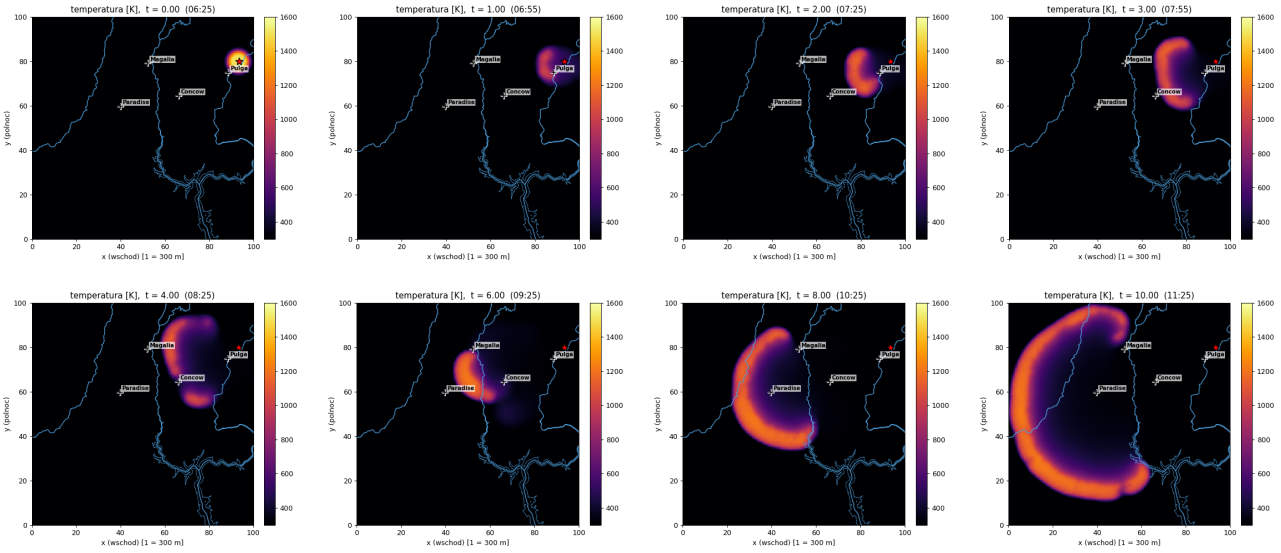
Obserwacje NIST do 11:25 objęte ogniem w symulacji	99,7 %
… najpóźniej 30 min po obserwacji	67,0 %
Mediana opóźnienia (symulacja − NIST)	+2 min
Spalona część palnego obszaru	48,6 %

Miejscowość	Symulacja	NIST
Pulga	6:33	6:46
Concow	8:10	7:24
Magalia	8:55	8:01
Paradise	9:40	10:46

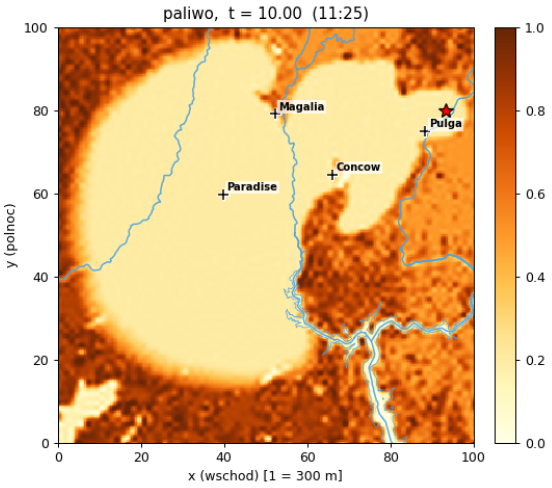
Tabela 3: Zgodność z NIST (lewa tabela) i pierwsze dojście ognia w promieniu 1,2 km od miejscowości (prawa tabela).

Kierunek i tempo pożaru zgadzają się z obserwacjami: mediana różnicy czasu dojścia ognia wynosi kilka minut, a symulacja obejmuje prawie wszystkie punkty NIST. Główne różnice:

- do Concow i Magalii ogień dochodzi ok. 45–55 min za późno, bo model nie ma żaru niesionego wiatrem, dzięki któremu ogień przeskakiwał kaniony;
- po 10:00 ogień rozchodzi się prawie kołowo po trawach, a spalony obszar jest większy niż w rzeczywistości;
- model nie uwzględnia rzeźby terenu, a wiatr jest z jednej stacji, więc taki sam w całym obszarze.



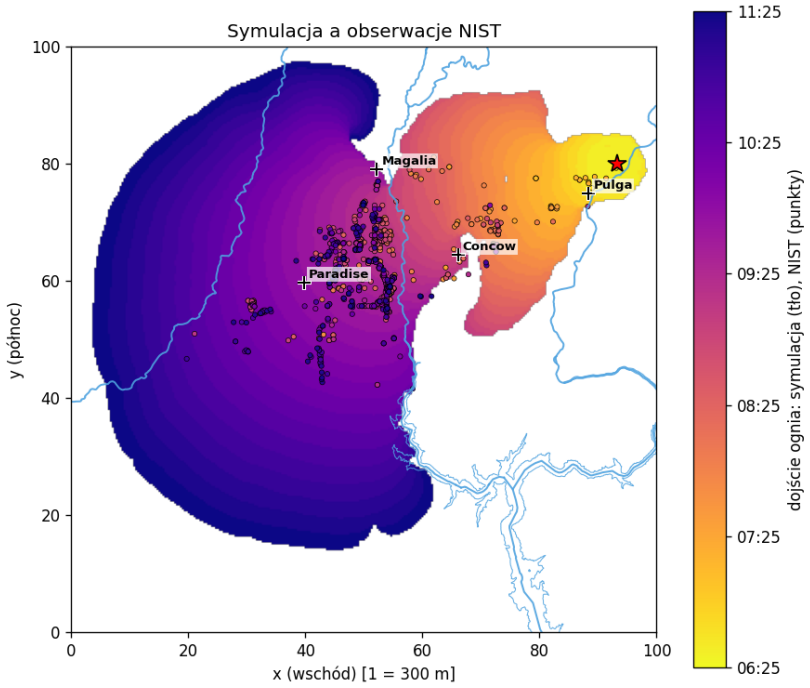
Rysunek 7: Temperatura co 30–60 min (kroki 0, 1000, 2000, 3000, 4000, 6000, 8000, 10 000).



Rysunek 8: Paliwo o 11:25. Ciemne obszary spłonęły.

Waga TU	Spalone	Objęte obserwacje NIST	Mediana opóźnienia
0.4	1,1 %	1,6 %	ogień gaśnie przy zapłonie
0.5	48,6 %	99,7 %	+2 min
0.6	78,9 %	100 %	−25 min

Tabela 4: Wpływ wagi podszytu leśnego (TU).



Rysunek 9: Czas dojścia ognia: tło pokazuje symulację, punkty obserwacje NIST (ta sama skala kolorów).

Literatura

[1] A. Maranghides i in., *A Case Study of the Camp Fire – Fire Progression Timeline*, NIST Technical Note 2135, 2021. Dane: `https://data.nist.gov/od/id/mds2-2954`

[2] LANDFIRE 2016 Remap, Fire Behavior Fuel Model 40, USGS/USFS, `https://www.landfire.gov`

[3] © współtwórcy OpenStreetMap (ODbL), `https://www.openstreetmap.org/copyright`

[4] Iowa Environmental Mesonet, stacja JBGC1, `https://mesonet.agron.iastate.edu`

[5] M. Łoś, IGA-ADS, `https://github.com/marcinlos/iga-ads`

A Kod

fire.hpp

```
// SPDX-FileCopyrightText: 2015 - 2024 Marcin Łoś <marcin.los.91@gmail.com>
// SPDX-License-Identifier: MIT

#ifndef FIRE_FIRE_HPP
#define FIRE_FIRE_HPP

#include <algorithm>
#include <cmath>

#include "ads/executor/galois.hpp"
#include "ads/output_manager.hpp"
#include "ads/simulation.hpp"
#include "ads/util.hpp"
#include "camp_fire_data.hpp"

// Camp Fire, 8 XI 2018, na kwadracie modelu [0, 100]^2: x na wschód, y na północ,
// 1 jednostka = 300 m (obszar 30 x 30 km). Dane z prawdziwych pomiarów, przeliczone przez
// przygotuj_model.py do camp_fire_data.hpp:
//   paliwo  - LANDFIRE 2016 FBFM40 (stan przed pożarem) + rzeki z OpenStreetMap
```

```
//   wiatr   - stacja RAWS Jarbo Gap (JBGC1), pomiary co godzinę
//   zapłon  - pierwsza obserwacja NIST TN 2135 (linia przesyłowa koło Pulga)
// Czas: t = 0 to zapłon o 6:25, 1 jednostka czasu = 0,5 h, więc t in [0, 10] to 6:25-11:25.

inline double falloff(double r, double R, double t) {
    if (t < r)
        return 1.0;
    if (t > R)
        return 0.0;
    double h = (t - r) / (R - r);
    return std::pow((h - 1) * (h + 1), 2);
}

// Liniowa interpolacja tabeli (xs rosnące); poza zakresem wartość z końca.
inline double interp_table(const double* xs, const double* ys, int n, double x) {
    if (x <= xs[0])
        return ys[0];
    if (x >= xs[n - 1])
        return ys[n - 1];
    int i = 1;
    while (xs[i] < x)
        ++i;
    double h = (x - xs[i - 1]) / (xs[i] - xs[i - 1]);
    return (1 - h) * ys[i - 1] + h * ys[i];
}

namespace ads::problems {

// Parametry do strojenia, nadpisywane z linii poleceń: fire <N> <p> <threads> [nazwa=wartość ...]
//
// Front ognia musi być szerszy niż element siatki (100 / N = 1,25), inaczej "zacina się" na liniach
// siatki i gaśnie albo idzie wzdłuż osi. Stąd rad, loss razy 6 i heat, burn przez 6 względem
// wartości ze slajdów (0.035 / 1.5 / 1 / 300): lokalne spalanie trwa 6 razy dłużej, ale progi
// zapłonu i gaśnięcia są te same, a front jest ok. 6 razy szerszy. Wartości dobrane porównaniem
// z obserwacjami NIST (porownaj_nist.py).
struct fire_params {
    double rad = 0.16;      // promieniowanie: delta_x = rad / eps; prędkość frontu rośnie jak sqrt(rad)
    double loss = 9;        // straty ciepła w górę: delta_z = loss * eps (mniej = większe straty)
    double cw = 0.5;        // jak mocno wiatr niesie ciepło
    double Tig = 800;       // temperatura zapłonu
    double fuel_min = 0.2;  // od ilu paliwa komórka się pali
    double heat = 1.0 / 6;  // mnożnik ciepła spalania Rc
    double burn = 50;       // tempo wypalania paliwa
    int fuel_heat = 1;      // 1 = ciepło spalania razy waga klasy paliwa (krzewy grzeją mocniej niż podszyt)
    double dt = 1e-3;
    double t_end = 10;      // 10 = 11:25
    int out_every = 250;    // zapis .data co tyle kroków (250 = ok. 40 klatek do animacji)

    // Wagi paliwa 0..1 dla klas LANDFIRE (mapa klas w camp_fire_data.hpp, podgląd: model_paliwo.png).
    // Pali się tylko paliwo > fuel_min; więcej paliwa = dłużej płonie. Przy fuel_heat = 1 waga
    // klasy mnoży też ciepło spalania Rc.
    // Przy heat = 1/6 ogień sam się utrzymuje od wagi ok. 0.55; niżej gaśnie (chyba że sąsiedzi grzeją).
    double SH = 1.00;     // krzewy (chaparral)
    double GS = 1.00;     // trawy z krzewami
    double GR = 0.80;     // trawy
    double TU = 0.50;     // podszyt leśny: wg NIST ogień go omijał (29% obszaru, 5% obserwacji)
    double TL = 0.90;     // ściółka leśna: sosny na grzbiecie Paradise, 45% obserwacji NIST
    double SB = 1.00;     // pozostałości po wycince
    double urban = 0.90;  // zabudowa: domy w Paradise i Magalia spłonęły
    double agri = 0.15;   // pola uprawne (nawadniane)
    double rock = 0.05;   // skały, piach
    double water = 0.0;   // woda (jezioro Oroville)
    double snow = 0.0;
    double other = 0.50;  // kody spoza listy
    double river = 0.55;  // pas 150 m przy rzece: min(waga klasy, river)

    double fuel_weight(unsigned char code) const {
        double w = other;
        switch (code & ~camp_fire::RIVER) {
        case camp_fire::URBAN: w = urban; break;
        case camp_fire::SNOW: w = snow; break;
        case camp_fire::AGRI: w = agri; break;
        case camp_fire::WATER: w = water; break;
        case camp_fire::ROCK: w = rock; break;
        case camp_fire::GR: w = GR; break;
        case camp_fire::GS: w = GS; break;
```

```
        case camp_fire::SH: w = SH; break;
        case camp_fire::TU: w = TU; break;
        case camp_fire::TL: w = TL; break;
        case camp_fire::SB: w = SB; break;
        default: break;
        }
        return code & camp_fire::RIVER ? std::min(w, river) : w;
    }
};

class fire : public simulation_2d {
private:
    using Base = simulation_2d;
    vector_type u, u_prev;

    vector_type fuel, fuel_prev;

    double ch = 1.0;
    double Ar = 5.7e-5;
    double rho = 1.293;
    double cp = 1.0;
    double cw;
    double kappa = 0.3;
    double xi = 2e-2;
    double sigma = 5.67e-8;
    double eps = 0.05; // "percentage error"
    double delta_x;
    double delta_z;
    double T0 = 300;
    double Tcomb = 1200;
    double Tig;
    double Ta = 300; // ???

    double M = 2;
    double M1 = 1;

    fire_params par;

    galois_executor executor;
    output_manager<2> output;

public:
    explicit fire(const config_2d& config, const fire_params& params, int threads)
    : Base{config}
    , u{shape()}
    , u_prev{shape()}
    , fuel{shape()}
    , fuel_prev{shape()}
    , cw{params.cw}
    , delta_x{params.rad / eps}
    , delta_z{params.loss * eps}
    , Tig{params.Tig}
    , par{params}
    , executor{threads}
    , output{x.B, y.B, 300} { }

    // Zapłon w miejscu pierwszej obserwacji NIST (6:25), gorący rdzeń ok. 0,75 km, otoczka do 2 km.
    double init_state(double x, double y) {
        double d = std::hypot(x - camp_fire::IGNITION_X, y - camp_fire::IGNITION_Y);
        return T0 + Tcomb * falloff(2.5, 7.0, d);
    }

    // Paliwo z siatki klas LANDFIRE: waga klasy z fire_params, interpolacja dwuliniowa.
    double fuel_map(double x, double y) const {
        const int n = camp_fire::FUEL_N;
        double gx = std::clamp(x / 100.0 * (n - 1), 0.0, n - 1.0);
        double gy = std::clamp(y / 100.0 * (n - 1), 0.0, n - 1.0);
        int i = std::min(static_cast<int>(gx), n - 2);
        int j = std::min(static_cast<int>(gy), n - 2);
        double hx = gx - i;
        double hy = gy - j;
        auto f = [this, n](int a, int b) { return par.fuel_weight(camp_fire::FUEL_CLASS[b * n + a]); };
        return (1 - hx) * (1 - hy) * f(i, j) + hx * (1 - hy) * f(i + 1, j)
             + (1 - hx) * hy * f(i, j + 1) + hx * hy * f(i + 1, j + 1);
    }
```

```
    // Wiatr ze stacji Jarbo Gap: jedna stacja, więc taki sam w całym obszarze.
    // Interpolacja liniowa między pomiarami co godzinę. Wieje z NE (ok. 30-40 st.),
    // czyli pcha ogień na południowy zachód; najsilniej rano, słabiej w południe.
    void wind_velocity(double /*x*/, double /*y*/, double t, double& bx, double& by) const {
        bx = interp_table(camp_fire::WIND_T, camp_fire::WIND_BX, camp_fire::WIND_N, t);
        by = interp_table(camp_fire::WIND_T, camp_fire::WIND_BY, camp_fire::WIND_N, t);
    }

private:
    void before() override {
        prepare_matrices();

        auto init = [this](double x, double y) { return init_state(x, y); };
        projection(u, init);
        solve(u);
        output.to_file(u, "out_0.data");

        auto fuel_init = [this](double x, double y) { return fuel_map(x, y); };
        projection(fuel, fuel_init);
        solve(fuel);
        output.to_file(fuel, "fuel_0.data");
    }

    void before_step(int /*iter*/, double /*t*/) override {
        using std::swap;
        swap(u, u_prev);
        swap(fuel, fuel_prev);
    }

    void step(int /*iter*/, double t) override {
        compute_rhs(t);
        solve(u);
        solve(fuel);
    }

    void compute_rhs(double t) {
        auto& rhs = u;
        auto& rhs_fuel = fuel;

        zero(rhs);
        zero(rhs_fuel);
        executor.for_each(elements(), [&](index_type e) {
            auto U = element_rhs();
            auto F = element_rhs();

            double J = jacobian(e);
            for (auto q : quad_points()) {
                double w = weight(q);
                auto p = point(e, q);
                double x = p[0];
                double y = p[1];
                double bx, by;
                wind_velocity(x, y, t, bx, by);
                double intensity = par.heat * (par.fuel_heat ? fuel_map(x, y) : 1.0);

                value_type u = eval_fun(u_prev, e, q);
                value_type fuel = eval_fun(fuel_prev, e, q);

                for (auto a : dofs_on_element(e)) {
                    auto aa = dof_global_to_local(e, a);
                    value_type v = eval_basis(e, q, a);
                    double inv = 1.0 / (rho * cp);

                    // Spline ringing near the ignition can drive T below 0. pow() and
                    // exp(-Ta/T) then overflow, and 0*inf becomes NaN for the whole field.
                    double T = std::max(u.val, 0.0);
                    double delta = T > Tig && fuel.val > par.fuel_min ? 1.0 : 0.0;
                    double hc = -70; // enthalpy
                    double r = 0.0;
                    if (delta > 0.0)
                        r = Ar * T * std::exp(-Ta / T);
                    double Rc = - intensity * 1e4 * rho * ch * hc * M / M1 * r;
                    double Qw = - rho * cw * (bx * u.dx + by * u.dy);
                    double qc = - kappa * grad_dot(u, v);
                    double qd = 0.0; // omitted
                    double qr = -4 * sigma * eps * delta_x * std::pow(T, 3) * grad_dot(u, v);
```

```
                    double Qconv = xi * (T0 - T);
                    double Qrz = sigma * eps / delta_z * (std::pow(T0, 4) - std::pow(T, 4));

                    double val = (Rc + Qw + Qconv + Qrz) * v.val + qc + qd + qr;
                    U(aa[0], aa[1]) += (u.val * v.val + steps.dt * inv * val) * w * J;

                    double fval = - delta * par.burn * r * fuel.val * v.val;
                    F(aa[0], aa[1]) += (fuel.val * v.val + steps.dt * fval) * w * J;
                }
            }
            executor.synchronized([&] { update_global_rhs(rhs, U, e); });
            executor.synchronized([&] { update_global_rhs(rhs_fuel, F, e); });
        });
    }

    void after_step(int iter, double /*t*/) override {
        auto i = iter  +1;
        if (i % par.out_every == 0) {
            std::cout << "Step " << i << std::endl;
            output.to_file(u, "out_%d.data", i);
            output.to_file(fuel, "fuel_%d.data", i);
        }
    }

    // double forcing(point_type x, double /*t*/) const {
    //     return 0;
    // }
};

}  // namespace ads::problems

#endif  // FIRE_FIRE_HPP
```

fire.cpp

```
// SPDX-FileCopyrightText: 2015 - 2024 Marcin Łoś <marcin.los.91@gmail.com>
// SPDX-License-Identifier: MIT

#include <cmath>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <map>
#include <string>

#include "fire.hpp"

int main(int argc, char* argv[]) {
    if (argc < 4) {
        std::cerr << "Usage: fire <N> <p> <threads> [nazwa=wartość ...]" << std::endl;
        std::exit(1);
    }
    int n = std::atoi(argv[1]);
    int p = std::atoi(argv[2]);
    int threads = std::atoi(argv[3]);

    // Parametry do strojenia (opis w fire.hpp, struct fire_params), np. rad=0.007 cw=1.5 TU=0.4
    ads::problems::fire_params par;
    std::map<std::string, double*> real{
        {"rad", &par.rad},   {"loss", &par.loss},         {"cw", &par.cw},
        {"Tig", &par.Tig},   {"fuel_min", &par.fuel_min}, {"heat", &par.heat},
        {"burn", &par.burn}, {"dt", &par.dt},             {"t_end", &par.t_end},
        // wagi paliwa
        {"SH", &par.SH},       {"GS", &par.GS},       {"GR", &par.GR},     {"TU", &par.TU},
        {"TL", &par.TL},       {"SB", &par.SB},       {"urban", &par.urban}, {"agri", &par.agri},
        {"rock", &par.rock},   {"water", &par.water}, {"snow", &par.snow}, {"other", &par.other},
        {"river", &par.river},
    };
    std::map<std::string, int*> whole{{"fuel_heat", &par.fuel_heat}, {"out_every", &par.out_every}};
    for (int i = 4; i < argc; ++i) {
        std::string arg = argv[i];
        auto eq = arg.find('=');
        std::string key = arg.substr(0, eq);
        if (eq != std::string::npos && real.count(key)) {
```

```
            *real[key] = std::atof(arg.c_str() + eq + 1);
        } else if (eq != std::string::npos && whole.count(key)) {
            *whole[key] = std::atoi(arg.c_str() + eq + 1);
        } else {
            std::cerr << "nieznany␣parametr:␣" << arg << "\nznane:";
            for (auto& [k, v] : real)
                std::cerr << '␣' << k;
            for (auto& [k, v] : whole)
                std::cerr << '␣' << k;
            std::cerr << std::endl;
            std::exit(1);
        }
    }

    // params.csv obok wyników (nagłówek + jeden wiersz): opis przebiegu, animate.py bierze z niego dt
    std::ofstream info{"params.csv"};
    info << "N,p";
    for (auto& [k, v] : real)
        info << ',' << k;
    for (auto& [k, v] : whole)
        info << ',' << k;
    info << '\n' << n << ',' << p;
    for (auto& [k, v] : real)
        info << ',' << *v;
    for (auto& [k, v] : whole)
        info << ',' << *v;
    info << '\n';

    // Camp Fire: kwadrat [0, 100]^2 = 30 x 30 km (x na wschód, y na północ).
    // t in [0, 10] to 8 XI 2018 6:25-11:25 (1 jednostka czasu = 0,5 h), patrz fire.hpp.
    ads::dim_config dim{p, n, 0.0, 100.0};
    int nsteps = static_cast<int>(std::lround(par.t_end / par.dt));
    ads::timesteps_config steps{nsteps, par.dt};
    int ders = 1;

    ads::config_2d c{dim, dim, steps, ders};
    ads::problems::fire sim{c, par, threads};
    sim.run();
}
```